

BrailleVision

An AI-powered assistive system that detects physical Braille from images or live camera input, reconstructs reading order, separates words correctly, converts the result into readable text, and speaks the output aloud.

98.0%
mAP@0.5

~24 ms
GPU inference time

26 / 26
Braille letters covered

FastAPI + YOLOv8
Deployment stack

1. Executive Summary

BrailleVision is a full-stack computer vision project designed to convert physical Braille into readable text and speech. The system combines a fine-tuned YOLOv8 object detector, custom spatial post-processing, a FastAPI backend, and a browser-based frontend. It supports image upload, live camera mode, annotated visual output, confidence-aware decoding, and text-to-speech.

A key technical challenge in Braille recognition is that correct translation requires more than classification of individual cells. The system must also reconstruct reading order across rows and identify word boundaries within the same row. BrailleVision addresses this using adaptive row clustering and horizontal gap-based word segmentation, so it can distinguish between separate words even when they appear on the same line.

Core innovation: the project does not stop at object detection. It adds a spatial translation layer that turns per-cell detections into valid, readable language output.

2. Problem and Goal

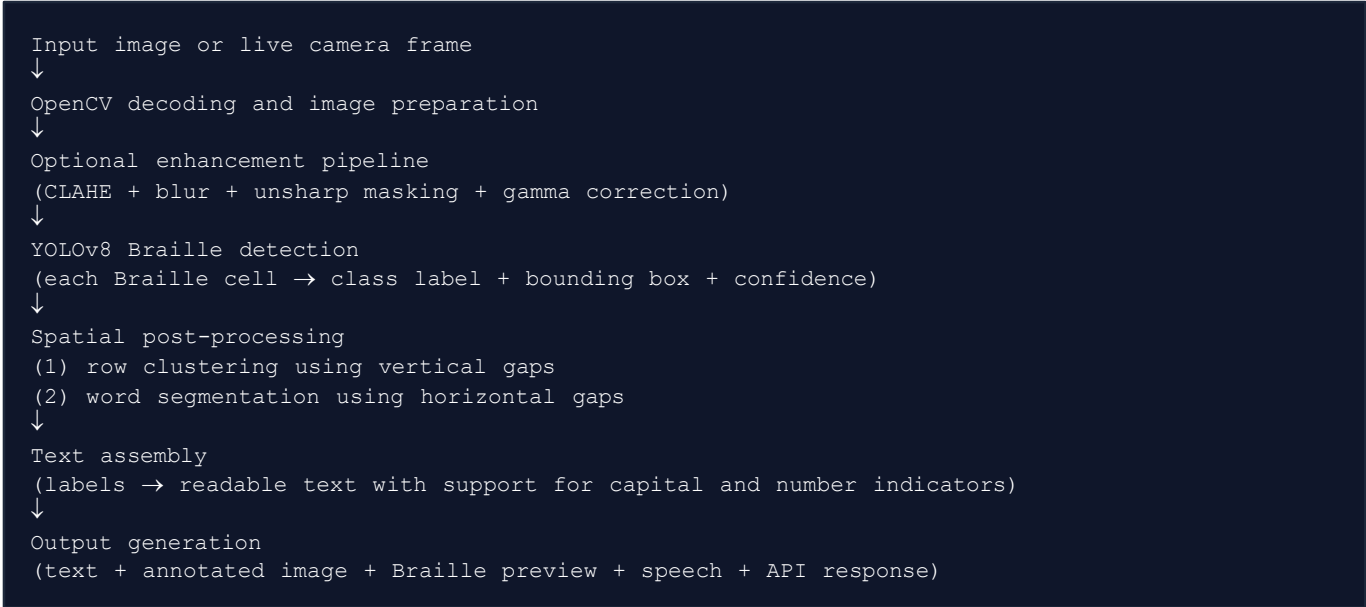
Problem

Braille is readable to trained users but inaccessible to most sighted people. Existing OCR tools are built for printed ink text, not raised-dot tactile patterns. Real Braille images also introduce uneven lighting, embossed surfaces, low contrast, camera perspective, and inconsistent spacing.

Goal

Build a system that can detect Braille letters accurately in real-world images, organize them in correct reading order, separate words correctly, generate accessible output, and present everything through an interface that judges and end users can understand immediately.

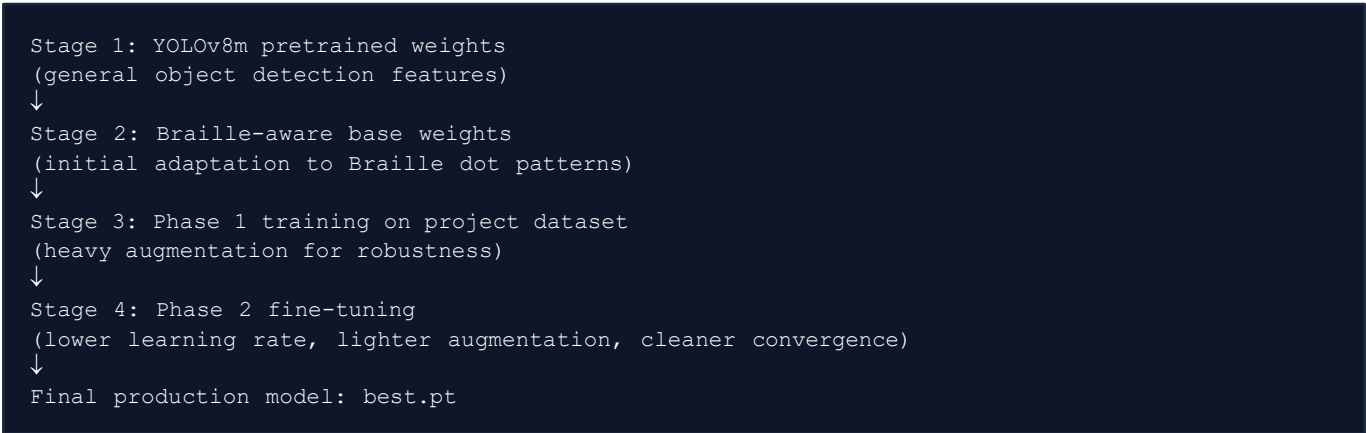
3. End-to-End Workflow



In upload mode, the user provides a Braille image through the web interface. The frontend sends the file to the backend, which runs detection and returns the result. In camera mode, the browser captures frames periodically and sends them to a lightweight endpoint so the user receives live updates with low latency.

4. Model Pipeline

The project uses transfer learning rather than training from scratch. This reduces training time and improves convergence because the model already starts with useful visual features. The final detector is based on YOLOv8m and recognizes 26 classes, one for each English Braille letter from A to Z.



Training Phase	Main Purpose	Characteristics	Outcome
Phase 1	Learn robust Braille features under	Low learning rate, heavy augmentation, larger search space	High robustness for noisy input features
Phase 2	Refine the model on the true target	Distilled learning rate, minimal augmentation, cleaner convergence	Improved final accuracy and more stable

The training script also includes practical engineering features such as checkpoint saving, resume support, automatic report generation, per-class analysis, and ONNX export for deployment.

5. Detection and Translation Logic

This section is the main technical strength of the project. The raw detector only predicts isolated Braille cells. To

make the result readable, BrailleVision adds a custom translation stage that reconstructs language structure from geometry.

5.1 Per-cell detection

Each YOLO prediction contains a bounding box, class label, and confidence score. The labels correspond to Braille letters. The detector does not directly predict full words or lines; instead, it predicts one cell at a time.

5.2 Reading-order row clustering

All detections are first sorted by vertical center. The system then computes the median box height and uses it to derive an adaptive row-splitting threshold. If the vertical gap between consecutive detections is larger than this threshold, a new row is started. This allows the same algorithm to work across different image sizes and camera distances.

5.3 Word segmentation within a row

A major issue in Braille recognition is that words in the same row can be incorrectly merged if the system only groups by row. BrailleVision solves this by measuring horizontal spacing between consecutive detections within a row. The algorithm considers box width, edge-to-edge gaps, and center-to-center distances. If a gap is significantly larger than normal letter spacing, it is treated as a word boundary. This makes the system robust to layouts where multiple words appear in the same line.

*Example: instead of reading the first row as one merged token such as **catdogmouse**, the updated logic can separate it correctly into **cat dog mouse**.*

5.4 Text assembly rules

After row clustering and word segmentation, the ordered labels are converted into final text. The code also supports semantic indicators such as capital indicator, number indicator, and explicit space token. This makes the output more aligned with real Braille reading rules instead of simple letter concatenation.

5.5 Robust inference for difficult images

The batch inference script extends standard prediction with a more advanced cascade. It can run multi-scale passes, ROI-based re-inference, tiled inference for high-resolution inputs, and enhancement-based passes. Duplicate detections are merged using custom non-maximum suppression that checks both overlap and normalized center distance. This improves recall while avoiding repeated predictions on the same Braille cell.

6. Output Generation

BrailleVision is designed to be useful both as a technical system and as an assistive interface. For that reason, it generates output in multiple formats.

Output Type	Description	
Readable text	Final translated text shown in the UI or CLI	
Annotated image	Bounding boxes, labels, and confidence scores drawn over	
Braille visualizer	2×3 dot representation for each detected letter in the browser	
Speech output	Web Speech API in browser, pyttsx3 in CLI mode	
Structured API respo	sJeSON with text, letter count, confidences, inference time,	

7. Technical Architecture

```
Frontend layer
- HTML, CSS, JavaScript
- image upload, webcam capture, result display, Braille preview, speech controls
↓
Backend/API layer
- FastAPI application
- file handling, decoding, preprocessing, inference endpoints, JSON responses
↓
Inference layer
- BrailleDetector in app.py
- YOLO model execution
- row clustering, word segmentation, text assembly
↓
Training and evaluation layer
- train.py for model development
- inference/predict.py for batch testing, robust inference, and export
```

File	Role in the system
app.py	Main inference core. Handles YOLO prediction, reading-order logic, text assembly, CLI image/video/camera
main.py	FastAPI backend. Provides endpoints for normal prediction, enhanced prediction, live frame prediction,
inference/predict.py	Batch inference and evaluation tool. Adds robust multi-pass inference, custom merging, rich annotation,
static/index.html	Frontend interface. Supports upload mode, live camera mode, confidence display, dot visualizer,
train.py	Training pipeline with two-phase strategy, checkpointing, resume logic, reporting, evaluation, and ONNX

8. Performance and Engineering Highlights

- 98.0% mAP@0.5 on the final model
- Approx. 24 ms inference time per image on GPU
- Support for real-time webcam mode and browser-based interaction
- Robust to low-contrast and embossed Braille through optional enhancement
- Word-level separation inside the same row, not just row-level grouping
- Production-oriented training pipeline with checkpoint backup and resume support
- Deployment-ready through FastAPI and export-ready through ONNX

9. Why This Project Stands Out

Many OCR systems stop at detection or classification. BrailleVision goes further by solving the full usability problem. It combines visual recognition, spatial reasoning, text reconstruction, accessibility output, and practical deployment in a single coherent system. The project is not just a model demo; it is an end-to-end assistive application with a clear user workflow, measurable performance, and extensible architecture.

10. Conclusion

BrailleVision demonstrates how deep learning can be turned into a practical accessibility tool. The project integrates object detection, adaptive spatial post-processing, rich visual output, speech generation, and web deployment into one system. Its most important contribution is the translation layer that reconstructs valid reading order and word boundaries from detected Braille cells. This transforms the project from a simple Page 6 classifier into a usable AI reading assistant.